NLP Assignment 1

Aug 2026

Assignment Guidelines

- We have a strict late policy: penalties increase to 10%, 20%, 50%, 100% for each day, where #days are rounded up.
- The assignment is mixed with theory and coding. You should submit a .zip containing your answer pdf and your code.
- The answer.pdf shoule be formatted using LaTeX, which includes answers for theory problems 1 to 4. If you meet problems using LaTeX, feel free to reach out to TAs.
- The full score is 100 points. For each subquestion, we will give partial score to partially correct answer. Double-check your submission to prevent typo and compilation error.
- You are free to discuss the problems with your classmates or search online, however, you must acknowledge the collaborators or online source at the end of the answer pdf.
- **You are not allowed to use LLMs** to assist you working on this assignment. If you find any typo or error, please ask us in the wechat group.

Problem 1 CFG (10 points)

Consider the following Context-Free Grammar (CFG) with the start symbol S:

```
S -> NP VP
NP -> Det Nom | PropN
Nom -> Noun | Noun Nom
VP -> Verb NP | Verb
Det -> 'the' | 'a'
Noun -> 'book' | 'flight'
PropN -> 'Houston'
Verb -> 'booked'
```

1. (5 point) Convert this grammar into Chomsky Normal Form (CNF). Show the steps of your conversion.

2. (5 point) Using the CNF grammar you derived in 1. , determine if the sentence "the book booked" is a valid sentence according to the grammar. Use the CKY parsing algorithm to fill out the parsing table. Show the completed table and state whether the sentence is accepted or rejected by the grammar.

Problem 2 Bigram Model (20 points)

1. (10 point) Consider the following corpus:

```
<s> I am Sam </s>
<s> Sam I am </s>
<s> I am Sam </s>
<s> I do not like green eggs and Sam </s>
```

   Using a bigram language model with **add-one (Laplace) smoothing**, what is the probability of the word `Sam` following the word `am`? That is, what is $P(\text{Sam}|\text{am})$?

   In your calculations, you should treat the sentence start marker `<s>` and the sentence end marker `</s>` as regular tokens in the vocabulary. The formula for add-one smoothed bigram probability is:

$$P(w_n|w_{n-1}) = \frac{C(w_{n-1}w_n) + 1}{C(w_{n-1}) + |V|}$$

   where $|V|$ is the size of the vocabulary.

2. (10 point) Now, suppose we build a model that does not use the end-of-sentence symbol `</s>`. Train an **unsmoothed** bigram model on the following training corpus:

```
<s> a b
<s> b b
<s> b a
<s> a a
```

   A language model should assign a probability distribution over the set of all possible sentences, meaning the sum of the probabilities of all sentences of all possible lengths should be 1.

Demonstrate that this unsmoothed bigram model (which lacks an end-symbol) fails to create such a distribution. You can do this by showing that:

- The sum of the probabilities of all possible 2-word sentences over the alphabet $\{a, b\}$ is 1.0.
- The sum of the probabilities of all possible 3-word sentences over the alphabet $\{a, b\}$ is also 1.0.

Problem 3 RLHF (30 points)

Recent advancements have seen the increasing application of capable AI systems in complex domains. For instance, Large Language Models (LLMs) have exhibited improved capabilities in multi-step reasoning and cross-task generalization in real-world deployment settings, and these abilities are strengthened with increased training time/data, and parameter size. However, various undesirable behaviors exhibited by advanced AI systems have raised concerns about the hazards from AI systems. Consequently, these concerns have catalyzed research efforts in AI alignment, which aims to make AI systems behave in line with human intentions and values, focusing more on the objectives of AI systems than their capabilities.

Here we introduce the basic idea of a representative approach for AI alignment, called **reinforcement learning from human feedback (RLHF)**. RLHF formulates the alignment during-training process as a bandit learning problem with reward signals. Let $\mathcal{X}$ denote the prompt set, which has a prior distribution $\mathcal{D}_x$, and the $\mathcal{Y}$ denotes the response set. Language models can thus be viewed as a policy $\pi : \mathcal{X} \to \Delta(\mathcal{Y})$, i.e., mapping each prompt to a probability distribution on $\mathcal{Y}$. RLHF assumes that there exists a reward oracle $r : \mathcal{X} \times \mathcal{Y} \to \mathbb{R}$, and aims to train a policy to generate a response $y \in \mathcal{Y}$ with a high reward $r(x, y)$ given a prompt x, i.e.,

$$\arg\max_{\pi} \mathbb{E}_{x\sim\mathcal{D}_x, y\sim\pi(\cdot|x)} r(x, y) \tag{1}$$

In practice, people often already have a base model π_{ref} before they conduct RLHF. To maintain model performance and mitigate the reward over-optimization issue[1], an additional KL-regularized penalty is added to the objective Eq.(10), and a more comprehensive objective is introduced:

$$\arg\max_{\pi} \mathbb{E}_{x\sim\mathcal{D}_x, y\sim\pi(\cdot|x)} \left[r(x, y) - \beta \mathrm{KL}\left(\pi(\cdot|x) \| \pi_{\text{ref}}(\cdot|x)\right)\right] \tag{2}$$

where $\beta \in \mathbb{R}_+$ is a hyperparameter. For simplicity we assume $\pi_{\text{ref}}(y|x) > 0$ for $\forall x \in \mathcal{X}, y \in \mathcal{Y}$.

Note: *A deep understanding of RLHF is not required to solve this problem. The solution relies only on mathematical derivation.*

[1] `https://arxiv.org/abs/2210.10760`

1. (10 point) If there are no restrictions on the parametrization of the policy π, show that the optimal policy for Eq.(2) is

$$\pi^*(y|x) = \frac{1}{Z(x)}\pi_{\text{ref}}(y|x)\exp\left(\frac{1}{\beta}r(x,y)\right), \tag{3}$$

where

$$Z(x) = \sum_{y'\in\mathcal{Y}} \pi_{\text{ref}}(y'|x)\exp\left(\frac{1}{\beta}r(x,y')\right). \tag{4}$$

Hint: *You can first prove for a specific x, then generalize to all x in X. A possible way is to formulate it as an optimization problem and use properties of KL-divergence.*

2. (10 point) In the practice of language models, could we directly calculate and use the optimal policy in Eq.(3)? Why? You only need to provide one reason in no more than 40 words.

3. (10 point) We assume that human preference distribution p^* satisfies[2]:

$$p^*(y_1 \succ y_2|x) = \sigma\left(r(x,y_1) - r(x,y_2)\right), \tag{5}$$

where σ is sigmoid function and human preference distribution means that conditioned on prompt x, response y_1 is favored over y_2 with probability $p^*(y_1 \succ y_2|x)$ by human annotator. Then given a human preference dataset $\mathcal{D} = \{(x^{(i)}, y_w^{(i)}, y_l^{(i)})\}_{i=1}^N$, which means that in the i_{th} sample, $y_w^{(i)} \succ y_l^{(i)}$ conditioned on $x^{(i)}$, the reward function r can be learned using a negative log-likelihood loss:

$$\mathcal{L}_r = -\frac{1}{N}\sum_{i=1}^{N}\left[\log\sigma\left(r(x^{(i)}, y_w^{(i)}) - r(x^{(i)}, y_l^{(i)})\right)\right]. \tag{6}$$

Based on this, a popular work[3] proposes to directly learn the policy π^* via parameter θ using a negative log-likelihood loss:

$$\mathcal{L}_\pi(\theta) = -\frac{1}{N}\sum_{i=1}^{N}\log\sigma\left(\beta\log\frac{\pi_\theta(y_w^{(i)}|x^{(i)})}{\pi_{\mathsf{ref}}(y_w^{(i)}|x^{(i)})} - \beta\log\frac{\pi_\theta(y_l^{(i)}|x^{(i)})}{\pi_{\mathsf{ref}}(y_l^{(i)}|x^{(i)})}\right). \tag{7}$$

Show your intuitions for deriving Eq.(7).

Hint: *Assume π_θ is a parameterization of the optimal policy π^*. Rearrange Eq.(3) to express the reward $r(x,y)$ in terms of π^* and π_{ref}. Substitute this expression into Eq.(6) to derive Eq.(7).*

Please choose exactly 2 problems from 4A, 4B, 4C, and 4D.

[2] https://en.wikipedia.org/wiki/Bradley-Terry_model
[3] https://arxiv.org/abs/2305.18290

Problem 4A Paper Reading (20 points)

Choose and read a paper from the list in `https://cloudygoose.github.io/nlp_papers.html`. Write a brief report (minimum length is 1 page, and maximum length is 3 page) that summarizes the paper's main contributions and your key takeaways (it's okay to copy-paste one or two figures from the paper). You may also include what you have learnt and questions you might have, etc. You do not need to understand everything or detail in the paper. Any reasonable report that meets the length requirement will receive at least 16 points.

Reminder:Please choose exactly 2 problems from 4A, 4B, 4C, and 4D.

Problem 4B Code Completion (20 points)

The goal of this problem is to code a script and train a small **skip-gram word2vec** model from scratch in PyTorch, using **negative sampling** for efficient training.

We have provided a Python script (`word2vec.py`) and a small text corpus (`corpus.txt`). Your task is to complete the four 'TODO' sections in the script.

Your model will learn two sets of embeddings for each word w in the vocabulary: a "center" vector (v_w) and an "outside" vector (u_w).

The model is trained using a negative sampling loss. For a given center word c, a true context word o (a positive sample), and K randomly drawn negative samples $\{w_1, \ldots, w_K\}$, your implementation should aim to minimize the following loss function:

$$\mathcal{L} = -\log(\sigma(u_o^T v_c)) - \sum_{i=1}^{K} \log(\sigma(-u_{w_i}^T v_c)) \,, \tag{8}$$

where $\sigma(x)$ is the sigmoid function.

You are required to complete the four 'TODO' sections in the `word2vec.py` file. Due to the small corpus size, we don't expect high-quality embeddings. We give scores based on the correctness of your implementation. Reasonable implementation will receive full 20 points.

1. (5 point) Define the `Word2Vec` model class. It must contain two layers: one for the center word vectors and one for the outside word vectors.
2. (5 point) Implement the negative sampling loss function as defined by the Eq. (8).
3. (5 point) Implement the logic to draw K random negative samples (non-context words) for each training pair.
4. (5 point) Complete the training loop.

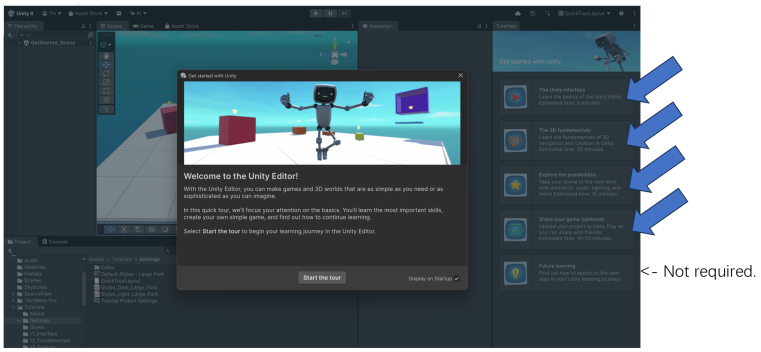


Figure 1: The embedded tutorial in Unity 6.

Note: *You are free to modify the provided code as needed to complete your implementation.*

Submission

Please submit your completed `word2vec.py` file in `problem_word2vec/`. You do not need to write anything in the report pdf.

Reminder:Please choose exactly 2 problems from 4A, 4B, 4C, and 4D.

Problem 4C Complete a Unity-6 Tutorial (20 points)

Download the unity hub, open Unity 6 and complete all sections (except "future learning") of the in-editor tutorial (Figure 1).

1. (10 points) Briefly explain (in less than 200 words) what are the most interesting stuff that you have learnt in this tutorial. Give a screenshot of the final robot game scene. You will get full points as long as your answer is reasonable.

2. (10 points) Spend some more time (around 45 minutes) in the robot scene, and try to use the provided materials to make this scene more interesting. Show screenshots (no more than 4 screenshots) of what you have done, and briefly explain your idea (no more than 150 words). You will get 6 points as long as your answer is reasonable. You do not need to submit code for this problem.

Reminder:Please choose exactly 2 problems from 4A, 4B, 4C, and 4D.

Problem 4D Basic Unity Questions (20 points)

Answer the following Unity-related questions **briefly** (we expect your answers to be short). You will mainly use natural language. But you are welcome to add (pseudo) code, math, or illustrations that might help.

If you find the problems difficult, the answers are contained in the book "Learning C# by Developing Games with Unity 2022, Seventh Edition" by Harrison Ferrone.

1. What type of GameObject are cubes, capsules, and spheres? (Chapter 6)
2. How do you convert a GameObject into a reusable prefab? (Chapter 6)
3. Which component adds real-world physics properties to a GameObject? (Chapter 7)
4. How do you create an instance of a prefab in the active scene? (Chapter 8)
5. How do you unsubscribe from an event? (Chapter 13)

Reminder:Please choose exactly 2 problems from 4A, 4B, 4C, and 4D.